

Available online at www.sciencedirect.com

ScienceDirect

journal homepage: www.elsevier.com/locate/coseComputers
&
Security

A self-protecting agents based model for high-performance mobile-cloud computing

Pelin Angin^{a,*}, Bharat Bhargava^b, Rohit Ranchal^c

^aDepartment of Computer Engineering, Middle East Technical University, Ankara, Turkey

^bDepartment of Computer Science, Purdue University, West Lafayette, IN, USA

^cIBM Watson Health Cloud, Cambridge, MA, USA

ARTICLE INFO

Article history:

Received 1 October 2017

Revised 14 April 2018

Accepted 16 April 2018

Available online 26 April 2018

Keywords:

Mobile-cloud computing

Tamper resistance

Autonomous agents

Performance

Security

ABSTRACT

Mobile-cloud computing (MCC) allows devices with resource and battery limitations to achieve computation-intensive tasks in real-time. While this new paradigm of computing seems beneficial for real-time mobile computing, existing MCC models mainly rely on keeping full clones of program code at remote sites and unstandardized/uninteroperable environments, hampering wider adoption. Moreover, the security risks arising from offloading data and code to an untrusted platform and the computational overhead introduced by complex security mechanisms stand as deterrents for adoption of MCC at large. In this paper, we present a context-dependent computation-offloading model for MCC, which is based on application segments packed into autonomous agents. This approach only requires isolated execution containers in the cloud to provide a runtime environment for the agents, and minimal involvement of the mobile platform during the computation process. The agents in the proposed model are able to protect themselves from tampering using integrity-checkpointing and an authenticated encryption-based communication mechanism. Experiments with two mobile applications demonstrate the effectiveness of the approach for high-performance, secure MCC.

© 2018 Elsevier Ltd. All rights reserved.

1. Introduction

Mobile computing devices have replaced desktops and mainframes for daily computing needs for the past decade. Despite the everyday advances in mobile computing technology, size restrictions impose limitations on the processing power and battery life of these devices, which limits their capabilities for real-time, computing-intensive applications such as image processing. Cloud computing offers the ability to fill the gap between the resource needs of mobile devices and availability of those resources, through the concept of *mobile-cloud computing* (MCC), which partitions mobile applications between

mobile and cloud platforms for execution, by dynamically offloading parts of mobile computation to cloud hosts.

Achieving high performance with mobile-cloud computing requires optimal partitioning of the mobile application components between the mobile and cloud platforms based on dynamic runtime conditions. Recent work on this problem has resulted in frameworks with various partitioning and optimization techniques. However, most of these frameworks impose strict requirements on the cloud side, such as a full clone of the application code or special application management software, hindering wide applicability in public clouds.

The other major obstacle for wider adoption of MCC is the security risks associated with sending sensitive data and code

* Corresponding author.

E-mail addresses: pangin@ceng.metu.edu.tr (P. Angin), bb@cs.purdue.edu (B. Bhargava), ranchal@us.ibm.com (R. Ranchal).

<https://doi.org/10.1016/j.cose.2018.04.011>

0167-4048/© 2018 Elsevier Ltd. All rights reserved.

to an untrusted platform, including but not limited to the following:

- Lack of control on resources and multi-tenancy of different users' applications on the same physical machine make cloud platforms vulnerable to attacks (Feng et al., 2011).
- In addition to privacy issues, programs running in the cloud are prone to tampering with code, data, execution flow and communication, as well as masquerading attacks (Chou, 2013).
- Mobile code can navigate through multiple platforms before returning to the origin, giving rise to the end-to-end security problem, which involves decreasing control with every further hop in the chain of platforms (Ali et al., 2015).

In order to provide complete security, the application should ensure all communication with/execution on the cloud platforms are trusted. Achieving ultimate flexibility and performance in mobile-cloud computing is contingent upon the availability of a secure computing framework capable of dynamic decision making with regards to the execution location of different program partitions. The performance requirements of real-time MCC call for a generalized computation offloading model that requires minimal involvement of the mobile platform for monitoring of offloaded computation, where all dynamic decision-making processes and integrity checks are achieved with lightweight components. The main challenges for achieving dynamic tamper resistance in MCC are the following:

- The tamper detection mechanism should involve minimal modification to the software structure, and be transparent to the programmer.
- Appropriate measures should be taken in case of tamper detection in a way that is transparent to the software.
- The runtime performance overhead of tamper detection and response should be minimal.
- Communication costs for detection and reporting of tamper should be minimal.
- The size of the program code/data should not be increased significantly due to the protection mechanism.

In this paper we present a mobile-cloud computation model based on autonomous agent-based application modules, which are augmented with performance checkpoints for self-performance evaluation and integrity verification units for dynamic detection and reporting of code tampering. The main contributions of this paper are as follows:

- We present an effective and easy-to-adopt dynamic computation offloading model for high-performance MCC.
- We present a dynamic performance estimation model that allows for tracking and relocation of offloaded computation to ensure optimal performance under varying mobile-cloud context.
- We propose a tamper resistance model for MCC that allows for dynamic integrity-checking and migration of offloaded computation to ensure trustworthiness of results received from a cloud platform.

- The proposed computation offloading and protection model relies on lightweight, transparent mechanisms for application code transformation/augmentation, allowing for low response time overhead and high interoperability.

The rest of this paper is organized as follows: Section 2 describes an autonomous agents-based computation-offloading model, namely AAMCC, for high-performance mobile-cloud computing. Section 3 presents experiment results for the application of the proposed model on two real-world mobile applications. Section 4 presents the proposed dynamic performance profiling model for allowing AAMCC agents to monitor their own performance during runtime and Section 5 provides the related experiment results. Section 6 provides the details of the proposed dynamic tamper resistance model for protecting the autonomous agents of AAMCC from malicious activity in the cloud. Section 7 provides a security analysis of the proposed tamper resistance model. Section 8 presents the results of experiments performed to evaluate the overhead of the proposed tamper resistance model. Section 9 reviews related work in high-performance MCC and MCC security. Section 10 concludes the paper.

2. Autonomous agents-based mobile-cloud computing (AAMCC)

In this section we present the details of an autonomous agents-based model for performance optimization in MCC.

2.1. Agent-based computation model

A *mobile (autonomous) agent* is a software program with mobility, which can be sent out from a computer into a network and roam among the computer nodes in the network (Chess et al., 1997). It can be executed on those computers to finish its task on behalf of its owner. Note that hereafter, we will be using the phrases *mobile agent* and *autonomous agent* interchangeably, as the agents in our proposed framework are both mobile and autonomous.

Fig. 1 shows a high level view of the agent-based MCC architecture, and Fig. 2 shows the model in action for a mobile application consisting of three modules. Each module of a mobile application in AAMCC is either an agent-based application module that is offloadable to the cloud (P2 and P3 in Fig. 2), or a native application component.

When a mobile application is launched, the *execution manager* contacts the *cloud directory service* to get a list of possible cloud hosts for offloadable application modules. After this step, an execution plan containing offloading decisions for the agent-based modules is created by the execution manager. If the execution plan requires offloading a particular application module, a bridge is formed between the caller of that module and the cloud host selected by the execution manager, through which the offloaded module migrates to the container in the host, carrying along its input parameters. Upon migration, the module starts executing and communicates its output data to the caller through the same bridge. In the case that the communication between the mobile platform and the cloud

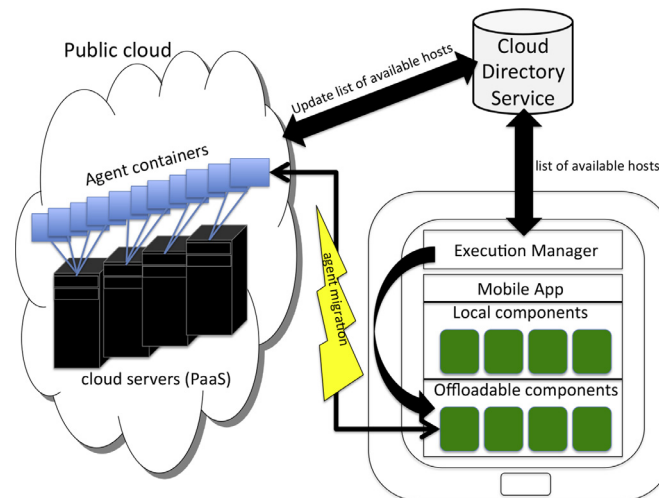


Fig. 1 – High level view of AAMCC architecture.

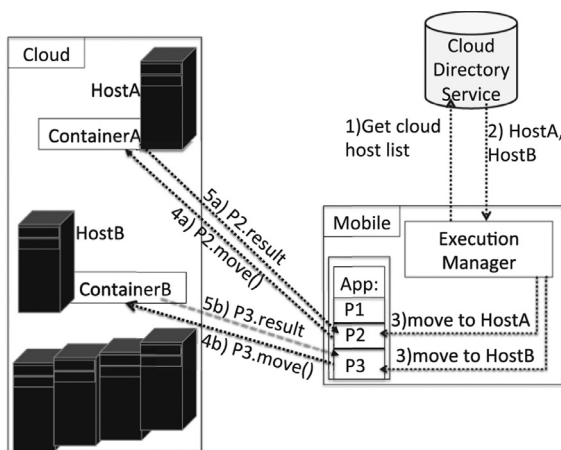


Fig. 2 – AAMCC in action for sample mobile application.

platform where the offloaded agent resides is disrupted before the mobile platform receives the computation result, the agent remains on the cloud platform until the mobile platform reconnects to request the result. The ability to reconnect to the agent provides high reliability for the model, obviating the need to execute the offloaded task again, to avoid performance penalties. In some cases it is possible that the connection to the cloud platform will be permanently disrupted due to platform failures etc. In order to avoid indefinite stalling in such a case, the execution manager enforces a timeout for remote partition execution, upon expiration of which a new agent is launched for the same task.

2.2. Model components

2.2.1. Autonomous application modules

An autonomous application module is a chunk of application code packed in an autonomous (mobile) agent that is executable on a cloud host. Autonomy of these application modules provides great advantages in the context of MCC due to the capability of transparently moving between mobile and cloud platforms without requiring management by their caller

and self-cloning to different virtual machines. Each mobile application in the framework consists of a set of autonomous agent-based application modules that are offloadable to the cloud for execution, in addition to a set of native application components that are always executed on the device due to constraints such as accessing native sensors of the device or providing the user interface of the application. Partitioning of the application into these two types of components is performed statically before installation of the application on the mobile device as described in Angin and Bhargava (2013). During the offline application partitioning process, any program partition that is not computation-intensive is not set as an offloadable component even if it does not have to be pinned to the device, as this would incur additional runtime processing overhead for partitions that would likely never be offloaded. In the prototype framework, these modules are implemented as JADE (Telecom Italia Lab) agents and they exhibit all features of a mobile agent (Chess et al., 1997) including stateful execution, asynchronous communication, autonomic decision-making and migration capability.

2.2.2. Cloud directory service

The cloud directory service is a web service that maintains an up-to-date database of agent containers available for use in the cloud. Along with the public IP addresses of the machine instances and the agent container identifiers, this database also keeps a record of machine instance architecture information and the number of currently active containers on each instance. This service is also responsible for the instantiation of new agent containers in case of need (invocation) by a mobile application and performance requirement-based allocation of cloud resources to different agents. The cloud directory service also maintains the service-level agreement (SLA) for each of the available instances. The execution manager on the device utilizes this information when making decisions regarding which cloud container to offload the modules to.

2.2.3. Cloud hosts

Cloud hosts (virtual machine instances) in the model provide platform as a service (PaaS) for agent-based application

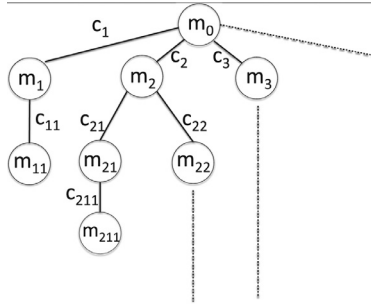


Fig. 3 – Execution tree structure.

partitions. A container-based service model is employed, where each container provides an isolated runtime environment for an offloaded application module to execute in. In the prototype framework, this component corresponds to JADE (Telecom Italia Lab) agent containers running on Amazon EC2 virtual machine instances (Amazon Web Services Inc.). Once an offloaded agent reaches the container selected by the execution manager as a result of solving the optimization problem described below and consulting the cloud directory service for the list of available agent containers, it starts (or if this is a relocation from another host, it resumes) execution of the application module contained in its code section. In addition to the regular containers for hosting the offloaded agents, each cloud host includes a main container, which has an agent management system controlling access to the platform and authenticating residing agents as well as controlling registrations, and a directory facilitator providing a yellow pages service to the agent platform. All containers on the host are registered with this main container for reachability from outside hosts. Each agent container is a multi-threaded execution environment composed of one thread for every agent plus system threads spawned by remote method invocation (RMI) runtime system for message dispatching. Each agent container is an RMI server object that locally manages a set of agents. It controls the life cycle of agents by creating, suspending, resuming and killing them. Besides, it deals with all the communication aspects by dispatching incoming messages and routing them according to the destination field. Upon completion of the execution of a specific agent, the agent is killed after communicating its output to the mobile device through the bridge established upon migration, and the container becomes available for hosting other agents.

2.2.4. Execution manager

One of the most important features affecting the performance of an MCC framework is its ability to adapt to changing network conditions when making offloading decisions. The offloading (execution) manager is a service running on the mobile device, which is responsible for dynamically determining whether a specific application module should be migrated to a cloud host to minimize total execution time, given the execution tree of the application. An execution tree, the structure of which is seen in Fig. 3, is a representation of the interactions between the modules of an application. The node at the top of the tree is the application entry node (i.e. the main module). If a node is the direct child of another node in the execution tree,

the module represented by the parent node makes a direct invocation to the module represented by the child node. In the case of cyclic dependencies between modules (which is not a good programming practice), the modules can be replicated in the execution tree, which will increase the depth of the tree, but will not require additional modifications to the solution of the optimization problem. Also, any application module invoked by multiple different modules is represented as a different node for each dependency (i.e. replicated), to avoid increased complexity of the offloading optimization problem.

Each node p in the execution tree is annotated with a number m_p . m_p is the time to execute the module (partition) p on the mobile device less the time to execute its submodules. Each edge in the tree is annotated with a number c_p , which is the total size of the data that flows into module p from its parent module and back for the invocation of p . Note that although the structure of the execution tree would not change between different executions, the edge annotations are determined at application runtime, as the execution time and size of data to be transferred between modules could be functions of the problem size. Note that this representation does not include numbers for execution time of a module in the cloud. Because the execution plan only includes offloadable modules, execution time in the cloud is considered negligible compared to that on the device. The limitations of this assumption can be overcome by augmenting agents with self-performance evaluation capability using a technique similar to the tamper-resistance mechanism described in Section 6.

The offloading manager uses the following cost model to make offloading decisions for each offloadable application partition (A submodule s of p is a module invoked by p):

td_p : on-device execution cost (time) of partition p

to_p : on-cloud execution cost (time) of partition p

b : the network bandwidth available to the application

p_s : the set of submodules of p .

Then td_p and to_p are calculated as follows:

$$td_p = m_p + \sum_{i \in p_s} \min(td_i, to_i) \quad (1)$$

$$to_p = c_p/b \quad (2)$$

In order to determine m_p , we currently use a static application profiler measuring the execution time of each application partition and record the results as metadata of the application.

As seen in Eq. (1), the optimization problem has a recursive formulation: To determine the execution cost of an application module, we first need to determine the execution costs of its submodules. However, the offloading decisions regarding the submodules of a module could become invalid once the costs of executing the module on the device and in the cloud are calculated, as offloading the module implies offloading its submodules too. We solve the optimization problem with a depth-first traversal of the execution tree, updating the local and offloaded execution cost of each module once all of its submodule execution costs are set. If the data transfer cost for a module exceeds its local execution cost, the module is kept on the device, otherwise it is offloaded. Once the whole execution tree is traversed using this algorithm, all children of a node chosen to be offloaded are offloaded with it as well

(even if the first decision for them during the traversal was to keep them local). Algorithm 1 shows how to solve the execution time optimization problem.

Algorithm 1: Algorithm for making offloading decisions.

```

input : tree: Execution tree, where each node is
        annotated with the local execution cost  $m$  and
        offloading data transfer cost  $o$ 
output: execution plan containing offloading decisions
        for each node in the tree
parentNode  $\leftarrow$  tree.root;
while parentNode has more children do
    childNode  $\leftarrow$  parentNode.nextChild;
    if childNode.nextChild = null then
        if childNode. $m$  > childNode. $o$  then
            childNode.decision  $\leftarrow$  offload;
            childNode.cost  $\leftarrow$  childNode. $o$ ;
        else
            childNode.decision  $\leftarrow$  local;
            childNode.cost  $\leftarrow$  childNode. $m$ ;
        end
        parentNode. $m$   $\leftarrow$  parentNode. $m$  + childNode.cost;
        if parentNode.nextChild = null then
            if parentNode. $m$  > parentNode. $o$  then
                parentNode.decision  $\leftarrow$  offload;
                parentNode.cost  $\leftarrow$  parentNode. $o$ ;
            else
                parentNode.decision  $\leftarrow$  local;
                parentNode.cost  $\leftarrow$  parentNode. $m$ ;
            end
            parentNode  $\leftarrow$  parentNode.parent;
        end
    else
        parentNode  $\leftarrow$  childNode;
    end
end

```

3. Performance of agent-based offloading

The performance of agent-based computation-offloading was evaluated with two mobile applications described below.

3.1. Experiments with face recognition

The first set of experiments was performed with a face recognition application based on the program at Darnok (2018), which given the picture of a person, identifies the most similar face to it in a set of pictures. This application assumes that the set of pictures to compare against is only locally available on the mobile device, i.e. if the application component processing the pictures is offloaded, the data for each picture is sent with it too. The experiments were run using an Android emulator¹

¹ Emulators utilizing x86 system images have been shown to provide better performance than some physical Android devices due to CPU acceleration.

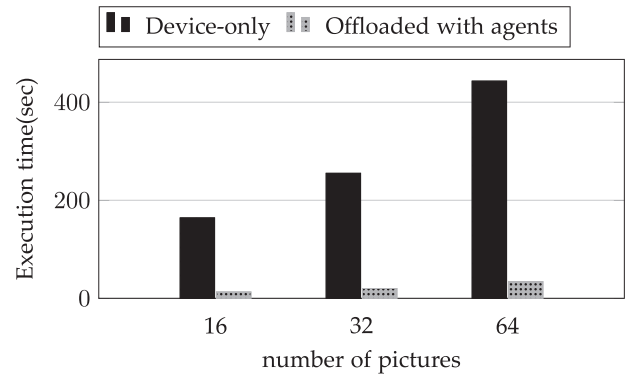


Fig. 4 – Execution time vs. number of pictures to compare against for face recognition application.

Table 1 – NQueens execution time (s).

# queens	Device-only	Offloaded	% difference
10	0.13	0.30	–130
12	3.07	0.61	80
13	18.65	1.77	90
14	123.45	9.46	92
15	872.01	60.60	93

over a period of 12 hours to capture possible variations in network conditions and the results reported are the averages for 75 runs. Fig. 4 provides a comparison of executing the application entirely on the mobile device vs. using the offloading framework, for different number of pictures (of 75 KB each) to compare against. We see that agent-based offloading achieves a significant 13 times shorter execution time than the device-only approach for all picture set sizes.

3.2. Experiments with NQueens puzzle

The second set of experiments were performed with the NQueens puzzle, which is the problem of placing n chess queens on an $n \times n$ chessboard such that no two queens can attack each other, i.e. no two queens share the same row, column or diagonal. In this set of experiments, we used an NQueens puzzle application, which finds all possible solutions to the puzzle for different number of queens using a recursive backtracking algorithm. The experiments were performed using a Motorola Atrix 4G (GSM Arena, Motorola Atrix, 2018), where the cloud host was a medium VMI in EC2. Each experiment was repeated 10 times and the average execution times for device-only execution, offloaded execution and the absolute time difference percentage (i.e. (device only - offloaded)/device only) for the two execution approaches are reported in Table 1. While execution times in seconds are close to each other for up to 12 queens, offloaded execution achieves significantly better performance than on-device execution for more than 13 queens and it outperforms on-device execution by about 15 times when the number of queens is 15.

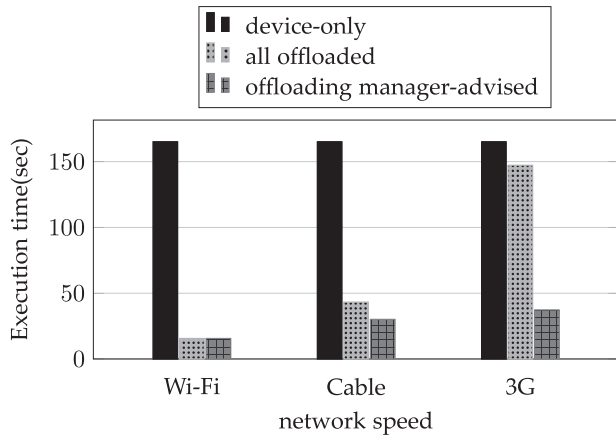


Fig. 5 – Comparison of the average execution times for a multi-module synthetic application in the case of (a) device-only execution, (b) complete offloading and (c) offloading manager-advised execution for different network speeds.

Table 2 – Connection speeds for offloading experiments.

Type	Downlink band-width	Uplink band-width	DNS delay	Downlink delay	Uplink delay
Wi-Fi	40 mbps	33 mbps	0 ms	1 ms	1 ms
Cable	6 mbps	1 mbps	3000 ms	2 ms	2 ms
3G	780 kbps	330 kbps	0 ms	100 ms	100 ms

3.3. Offloading manager performance

The performance of the offloading manager was evaluated with a synthetic application consisting of 6 offloadable modules obtained from parts of the NQueens puzzle application. Each module in the application has a different execution time on the mobile device and different amount of data transfer requirements for offloading. Fig. 5 shows a comparison of the total execution times of the application for the cases of (a) monolithic execution on the device, (b) complete offloading to the cloud, and (c) using the offloading manager to decide which modules to offload to the cloud, where the device's available bandwidth is upper-bounded by the connection speed types on the x-axis. The values for the different connection types is provided in Table 2. The experiments were run on a Motorola Atrix 4G device for each connection speed type.

As seen in Fig. 5, using the proposed offloading manager for execution platform decisions always achieves better (or same) performance than both monolithic execution on the device and offloading all modules to the cloud.

4. Dynamic performance profiling

The ability to make a close estimation of the *makespan* (the time taken to run a program segment) of an application module is important for an MCC model to be effective. While most

of the previously proposed makespan estimation models for MCC are based on the assumption that the execution time of an application module offloaded to the cloud will be negligible, this assumption may not always hold due to the dynamic nature of cloud platforms and the lack of information about the platform characteristics in some cases. In previous work (Angin et al., 2015), we showed that the characteristics of cloud hosts can have significant effects on the performance of mobile applications. Additionally, because cloud resources are shared between multiple users, the existence of other users on the same platform can significantly affect the makespan of an application in the cloud as well.

Estimation of the makespan of an application module serves the following purposes in the context of MCC:

- The estimated makespan can be compared to an expected value to see whether the execution is likely to satisfy specific time constraints. In the case of a low-performance platform, the computation can be migrated to a different platform.
- In the case of continuous or periodic makespan estimation updates, fluctuations in the estimates can provide an idea about the sharing of resources on the platform.

4.1. Makespan estimation model

Software execution profiling is an active area of research, as runtime performance of applications is of utmost importance for many systems. Although profiling the execution of an application under stable conditions on a specific architecture can yield accurate estimations for the makespan of the application under the exact same conditions, getting an accurate estimation under highly dynamic conditions and for platforms with different architectures is much more difficult. One of the main approaches used for makespan estimation is low-level analysis of a program to identify all instructions in the program and calculate the number of CPU cycles the program would take to execute, which then translates into a specific total execution time based on the architecture of the platform (Bate et al., 2000). However, this approach is too specific and requires knowledge of the exact architecture of the target platform. It also does not account for the dynamicity of the runtime environment. In order to achieve accurate results, a makespan estimation model should continuously update its estimates based on changing runtime conditions, and not rely on a specific architecture. Fitness for MCC also requires that the model not be too complex for real-time requirements.

The model we propose for makespan estimation of an application module is a simple, platform-independent model based on heuristics affecting makespan. The main idea is that the relative makespans of different execution blocks in an application should be fixed under stable runtime conditions. This means that, if it takes x seconds to execute a code block b_1 on a machine, and $2x$ seconds to execute block b_2 on the same machine, then b_2 will take $2y$ seconds to execute on another machine where b_1 takes y seconds. Exceptions are:

- Code blocks involving network activity, since their makespan is highly dependent on network conditions.

- Code blocks involving conditionals with branches of significantly different computational complexity.
- Code blocks accepting input parameters of significantly different sizes at runtime (at different runs), where the input size determines computational complexity.

The makespan estimation model works as follows:

1. The list of exceptions above forms the list of heuristics we use in the model, i.e. when selecting the code blocks whose makespans are to be integrated into the estimation model, we eliminate such blocks.
2. For the remaining code blocks, we run the application multiple times on the same platform and record the average makespan for each block.
3. Performance checkpoints corresponding to the selected code blocks are inserted into the application.
4. During application execution, each performance checkpoint records the actual makespan of the corresponding code block at runtime. It also makes an estimation of the makespan of the next code block with a performance checkpoint, based on the relative makespans of the two code blocks.

For an application with n performance checkpoints, the makespan for code block number x is estimated as

$$T_{est}(x) = T_{true}(x-1) \times \frac{T_{avg}(x)}{T_{avg}(x-1)} \quad (3)$$

In Eq. (3):

T_{est} : Estimated makespan for a code block

T_{avg} : Average makespan for a code block (from statistics)

T_{true} : Actual runtime makespan for a code block

The estimated total makespan for the remaining code blocks after x is calculated using Eq. (4):

$$T_{remaining}(x) = \sum_{i=x+1}^n T_{true}(i) \times \frac{T_{avg}(i)}{T_{avg}(x)} \quad (4)$$

4.2. Performance checkpoints integration

In order to be used in the proposed framework, the makespan estimation model needs to be integrated into the autonomous application modules, so that they can monitor their own performance during execution on a cloud platform and take appropriate actions based on the estimated remaining execution time. The integration is performed using aspect-oriented programming (AOP), which provides complete transparency to the underlying application code. In the case of self-performance evaluating agents, an updated estimate of the remaining makespan is calculated at each performance checkpoint during agent execution. The agent uses this information to clone itself to a different platform if the estimate of the remaining makespan is higher than an expected threshold ϵ . Note that the agent code is transformed into a stateful program to use this cloning approach, so that the time spent on the already executed code blocks is not wasted. States correspond to the performance checkpoints.

Table 3 – Makespan estimation errors for different VMIs.

Instance type	Avg. true makespan (ms)	Avg. absolute estimation error (ms)	Absolute error percentage (%)
m1.small	15,142	292	1.9
c1.medium	10,147	125	1.2
m1.medium	7277	132	1.8
m1.large	7234	75	1.0

4.3. Cloud host relocation

Allocation of agent containers by the cloud directory service is a significant component for the performance of the proposed MCC model, as it not only determines the first host an application module is assigned to, but also the hosts it might relocate to upon facing lower than expected performance. An optimal allocation scheme will also ensure cloud resources are used at full potential and costs are kept at minimum. In order to achieve optimal placement, the cloud directory service utilizes performance checkpointing data from autonomous agents. The following rules are used when assigning agents to cloud hosts:

- If an agent relocates, it is always assigned to a host of higher computation power or fewer active containers.
- If an agent is within a specified percentage of its threshold makespan, no new agents are assigned to its host until it terminates execution or leaves the host.
- Container assignments start with small instances.

The SLA information maintained by the cloud directory service is utilized for the optimal selection of the hosts during relocation in order to ensure that the performance will not fall below certain thresholds upon migration.²

5. Performance profiling experiments

We evaluated the performance of the makespan estimation model with the face recognition application running on different machine instances in EC2. 9 performance checkpoints were inserted into the application, which correspond to the major method calls. Each experiment was repeated 100 times and we report the average numbers over those runs. Table 3 summarizes the average of the absolute errors and the ratio of the average absolute error to the average true makespans. As seen in the table, the average absolute error is less than or equal to 292 ms, which translates into less than 2% of the total makespan for all instance types, which is negligible for most computation tasks. We also observe that the error increases slightly on instances with smaller computing power.

² While a vertical scaling solution is utilized in this work when making relocation decisions due to unfavorable performance, a horizontal solution that uses multiple instances of the same/small size can also be utilized if the task is parallelizable. This would require a minor modification to the agent code for special marking of parallelizable tasks.

Table 4 – Average execution time savings (in seconds) of self-performance evaluation for face recognition module.

m1.large	c1.medium	m1.medium	m1.small
78.70	74.90	71.94	62.04

These results prove that the proposed performance profiler is an effective tool for MCC.

We also performed experiments to see the effect of integrating the proposed self-performance evaluation approach into autonomous application modules. Experiments were performed with the face recognition module always offloaded to a micro instance in EC2. Based on the performance evaluation on the micro instance, the offloaded agent made the decision regarding whether to clone itself to another container in EC2. For the experiments, a CPU-intensive C process was running on all instances in addition to the face recognition module.

Table 4 shows the average execution time savings obtained by using the self-performance evaluation and cloning approach, for different types of machine instances as the second hop (the host where the agent is cloned to). The average total makespan without dynamic performance monitoring in the experiments was around 101 seconds. As seen in the table, the proposed model is able to achieve between 61% - 73% time savings by making the decision to clone an agent under unfavorable runtime conditions.

6. Self-protecting agents model for tamper resistance

In this section we present our proposed tamper resistance model for AAMCC. We start with a brief background on preliminary concepts related to the theory and implementation of the proposed model.

6.1. Preliminaries

6.1.1. Tamper resistance

Tamperproofing a program is ensuring that it *executes as intended*, even when there is an adversary who tries to disrupt, monitor or change the execution. An attacker typically modifies a program with the intent to force it to execute in a way to serve malicious purposes. The attacker can achieve this in one of the following ways (Collberg and Nagra, 2009):

- Removing code from and/or inserting new code into the executable file before execution
- Removing code from and/or inserting new code into the program during execution
- Altering the runtime behavior of the program using external agents such as emulators, debuggers, or a hostile operating system

Cloud computing platforms are especially vulnerable to tampering due to the multi-tenancy of applications from different parties on the same physical machine. In a public cloud, it is hard to control the intentions and actions of the different

users of a platform. A security case study of the Amazon EC2 by Ristenpart et al. (2009) has revealed the vulnerabilities introduced by virtualization, which is the main technology most cloud computing providers rely on to provide shared access to the same physical infrastructure. According to the study, it is possible for an attacker to identify where a particular virtual machine is likely to reside and launch virtual machines on the same physical machine as the target. Tamperproofing of code executing in the cloud is especially important to mitigate the effects of such targeted attacks.

Remote tamper resistance is a special case of tamper resistance, where the program to be protected runs on an untrusted platform, but communicates with a master program on the trusted platform, which detects or is made aware of any tampering with the program running on the untrusted platform (Collberg and Nagra, 2009). This kind of tamper resistance is especially necessary in the context of MCC, where the mobile platform needs to ensure the integrity of offloaded computation before integrating results/data received from the cloud platform.

One of the main techniques used for remote tamper resistance is the use of introspection for the program running on the untrusted platform and reporting the hash value of the code to the trusted platform, which can make sure that it matches a pre-computed value. *Introspection* is the examination of a program or code segment to check that its integrity is preserved. The main method used for code introspection is the augmentation of a program with code that computes a hash value (or a checksum) over a code region and compares it to an expected value (Collberg and Nagra, 2009). Attacks against tamper-resistant software using introspection generally use pattern-matching methods to identify locations in the program that include suspicious code (such as conditionals checking the value of a hash).

6.1.2. Authenticated encryption

Authenticated encryption (AE) is a shared-key based transform whose goal is to provide both *privacy* and *authenticity* of the encapsulated data (Bellare and Namprempre, 2008; Katz and Yung, 2001). The encryption process E in an authenticated encryption scheme takes a plaintext p and a shared secret key k and returns a ciphertext c , whereas the decryption process D takes the ciphertext c and the same key k and outputs either the plaintext p or a special symbol $\perp$, i.e.

$$\begin{aligned} E(k, p) &\rightarrow c \\ D(k, c) &\rightarrow p \cup \perp \end{aligned} \quad (5)$$

An output of $\perp$ by D means that the received ciphertext c was invalid or unauthentic. Using AE for a message sent by Alice to Bob, encrypted with their shared key k has two implications:

1. An active adversary (man-in-the-middle) cannot create a ciphertext which would make Bob believe that it was encrypted by Alice, i.e. if $D(k, c) \neq \perp \Rightarrow c$ is from someone who knows the shared key k .
2. The message is protected against chosen ciphertext attacks, i.e. the privacy of the message is preserved.

6.2. Proposed tamper resistance model

6.2.1. Tamper detection and reporting protocol

The approach we propose for tamper-resistant execution of offloaded code in the cloud is based on augmenting the mobile agents sent to a cloud platform with self-protection capability, using aspect-oriented programming (AOP) based integrity checkpoints distributed throughout the agent code to ensure timely detection of tamper. Upon tamper detection, the agent stops execution, moves to a different platform and resumes execution from the last integrity-verified checkpoint. The self-protection method relies on the following two main ideas:

1. **Tamper checking guards:** As the agent code is executing in the cloud, the integrity of the code is checked using introspection by *software guards* placed at various *integrity checkpoints* (endings of consecutive code blocks). These guards are also capable of reporting tamper to the mobile platform.
2. **Guard tamper tracking code:** Each time a software guard checks for tamper, its own hash value is saved in agent data. The agent accumulates the guard hash values in a variable, which forms the key to encrypt the result sent to the mobile platform when the computation is complete. The mobile platform is capable of computing the key value independently, therefore it can decrypt the message received to extract the result from the autonomous application module.

Using an encryption key formed by the dynamic hash values of the software guard code in the authenticated encryption of the result has the following implication: For the correct encryption key to be formed, the guard hash value at each integrity checkpoint needs to be integrated into the key and the integrated value needs to match the true (original) hash value for that guard. If the result received from the mobile agent cannot be decrypted by the mobile platform, there are two possibilities:

1. The message containing the result was modified in transit to the mobile platform.
2. The message was not encrypted with the correct key on the cloud platform.

In either of these conditions, the result cannot be integrated into the program on the mobile platform.

In the proposed security approach, accumulation of guard hash values during agent code execution is a stealthy tamper resistance mechanism in the sense that it does not involve any checks (conditional expressions) that are prone to detection by pattern matching methods used by attackers. It is important to form the encryption key using all guard hash values, as we are interested in detecting any tamper during the execution of the agent. This incremental formation of the encryption key serves two purposes:

1. The encryption key is not revealed until the end of program execution, which makes it harder for an attacker to dynamically swap the value of the corresponding variable with the original key value to hide tamper.

2. The mobile platform can ensure that all integrity verification blocks in the offloaded module were executed in the case of a correct key value, as the execution of guard hashing code is triggered by the execution of guard code blocks, which allows for checking the authenticity of the results received.

As evident from the above description, the strength of the mechanism for detecting tampering with the software guards relies heavily on the strength of the encryption key. In order to create a strong key, allowing for the detection of tamper with high probability, the process of forming the key should satisfy the following conditions:

- All guard hash values should be integrated into the key.
- The individual guard hash values should be formed using a collision-resistant hash function.
- The function forming the encryption key from the individual guard hash values should output uniformly distributed (pseudorandom) results in its output domain.
- The function forming the encryption key should be able to operate incrementally, instead of requiring all input at once.

Let H be a collision-resistant hash function used to calculate the guard hash values. The algorithm to compute the encryption key is provided in [Algorithm 2](#).

Algorithm 2: Encryption key generation.

```

input : guardCode: guard bytecode array
         $H$ : secure hash function
         $F$ : guard hash accumulation function
output: encryption key for autonomous application
        module result

numBlocks  $\leftarrow$  guardCode.size;
key  $\leftarrow$  0;
mostRecentGuard  $\leftarrow$  0;
while mostRecentGuard < numBlocks do
    | hashOfGuard  $\leftarrow H(\text{guardCode}[\text{mostRecentGuard}]);$ 
    | key  $\leftarrow F(\text{key}, \text{hashOfGuard});$ 
    | mostRecentGuard  $\leftarrow \text{mostRecentGuard} + 1;$ 
end

```

Another condition that needs to be satisfied for the tamper detection mechanism to operate correctly is the ability of the mobile platform to calculate the encryption key independently, as a symmetric-key encryption mechanism is used in the model. This means that integrity checkpoints should be placed in the code in such a way that their execution is independent of the control-flow of the program, i.e. the corresponding guard code is executed no matter which path the program takes based on the runtime conditions. Furthermore, care should be taken to use encryption parameters that differ from the previous run of the same code in the cloud, to prevent attacks where the encryption key and initialization vector (IV) could have been saved and reused by malicious parties. This issue is handled by random generation of an IV on the mobile platform before each agent offload. Additional security can be achieved by introducing a mechanism that random-

izes the values of guard hashes integrated into the encryption key through insertion of random bytes into the code with no effect on guard operation for different runs of the agent. Algorithm 3 below describes the setup process run by the execution manager for offloading an agent to a cloud host.

Algorithm 3: Offloading setup.

```

input : guardCode: guard bytecode array
        H: secure hash function
        F: guard hash accumulation function
output: Autonomous agent configuration

 $IV \leftarrow \text{random}(1, 2^{64})$ ;
 $\text{agentState}.IV \leftarrow IV$ ;
 $\text{guardsCovered} \leftarrow 0$ ;
while  $\text{guardsCovered} < \text{guardCode.size}$  do
     $\text{guardCode}[\text{guardsCovered}] \leftarrow$ 
         $\text{guardCode}[\text{guardsCovered}].\text{append}(\text{randomBytes})$ ;
     $\text{guardsCovered} \leftarrow \text{guardsCovered} + 1$ ;
end
 $\text{agent.guardCode} \leftarrow \text{guardCode}$ ;

//Compute secret key asynchronously
 $\text{key} \leftarrow 0$ ;
 $\text{guardsCovered} \leftarrow 0$ ;
while  $\text{guardsCovered} < \text{guardCode.size}$  do
     $\text{hashOfGuard} \leftarrow H(\text{guardCode}[\text{guardsCovered}])$ ;
     $\text{key} \leftarrow F(\text{key}, \text{hashOfGuard})$ ;
     $\text{guardsCovered} \leftarrow \text{guardsCovered} + 1$ ;
end

```

Fig. 6 shows the UML activity diagram of the execution life-cycle of a mobile agent in the proposed tamper resistance framework. The main steps taken by the agent during execution are as follows:

- Step 1: The running *guard hash* and *agent state* values for mobile agent MA are reset.
- Step 2: MA is sent to a selected VMI for execution.
- Step 3: The integrity of the code and data of the MA is checked before starting execution on the cloud platform.
- Step 4: If the integrity of MA was preserved during the transfer, MA starts executing the first code block in the application module and goes onto Step 5. Otherwise, a new VMI *V* is selected by inquiring the *cloud directory service*, a new MA is sent to *V*, and execution restarts from Step 1.
- Step 5: The software guard for the most recently executed code block checks the runtime integrity of the block. If no tamper was detected, the running guard hash value is updated with the hash of the guard code performing the most recent check and the agent state is updated to the last integrity checkpoint. Step 5 is repeated until there are no more code blocks to execute. If tamper was detected by the guard, a new VMI *V* is selected by inquiring the *cloud directory service*, MA sends a message to the mobile platform including the tamper report and its new VMI address, moves to *V* and resumes execution from the last tamper-free execution checkpoint.
- Step 6: The result of the offloaded computation is encrypted with the guard hash value and sent to the mobile platform.

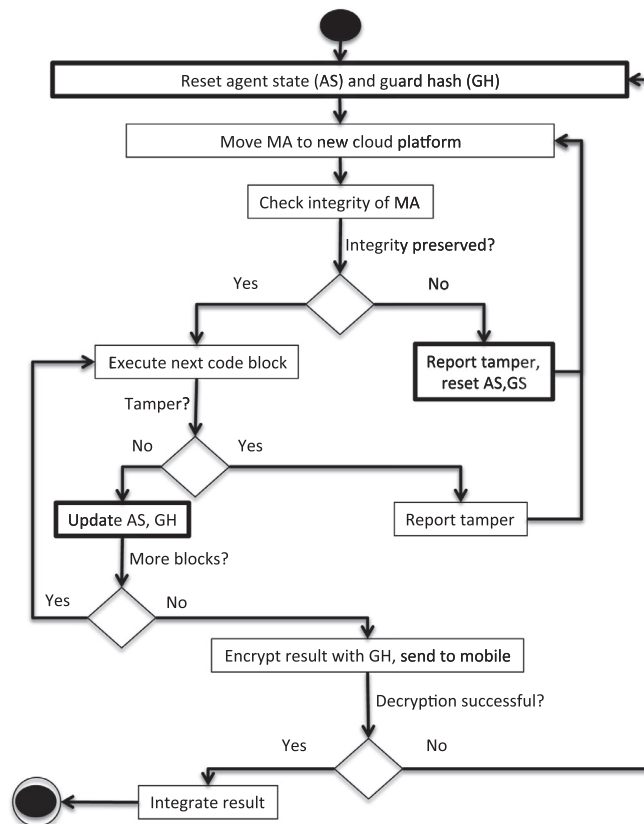


Fig. 6 – Activity diagram for tamper resistance mechanism.

- Step 7: If the mobile platform is able to successfully decrypt the result message received from MA, it integrates the result into the program. If the decryption of the result message fails on the mobile platform, the whole process of MA's execution is repeated starting with Step 1.

6.2.2. Implementation details

In the implementation of the proposed tamper resistance model, integrity checkpoints are inserted around every method call using the [AspectJ](#) AOP ([Påhlsson, 2002](#)) framework. AOP in general allows for injection of code at three different points: (a) at run-time through a dynamic proxy (b) at load-time when the target classes are being loaded (this happens at byte-code level for Android) (c) at build-time by modifying the compiled classes before packaging. The code injection process is completely transparent to application development. Advices (the code injected into the class files, i.e. the guard code) do not require direct access to the source code of the class. It is possible to insert advices around every method call in every class using regular expressions to define the pointcuts (locations to insert the advices).

The code snippet in [listing 1](#) shows how to implement a guard using the [AspectJ](#) AOP framework for the methods of an application class file *QueensBehaviour*. Line 2 in the code declares a pointcut *methodCalls*, which is a regular expression to match during the execution of the program. In this case, the pointcut matches all invocations of all methods of the *QueensBehaviour* class. Line 5 declares the code (advice) to be executed upon matching the declared pointcut. The *around* keyword specifies that the enclosed code has parts to be executed before (lines 8 – 18) and after (lines 21 – 28) the body of the

matched methods, with the body of the method executing in between (as specified by the special method *proceed*). We see that the whole process of adding integrity checking functionality is completely transparent to the actual application code.

The hash value for each method is calculated on the byte-code for that method using the SHA-256 secure hash algorithm, and the encryption key is formed using a bitwise XOR of the individual guard hash values. The Galois/Counter (GCM) mode of operation ([Grew and Viega, 2004](#)) is used for the authenticated encryption of the result by the mobile agent in the cloud platform and the decryption on the mobile platform. GCM is a block cipher mode of operation that uses universal hashing over a binary Galois field to provide authenticated encryption. It can be implemented in hardware to achieve high speeds with low cost and low latency, and software implementations can achieve very high performance by using table-driven field operations. Authenticated encryption using GCM takes three inputs:

- A secret key *K*
- An initialization vector *IV*, that can have any number of bits between 1 and 2^{64} . For a fixed value of the key, each *IV* value must be distinct, but need not have equal lengths.
- A plaintext *P*, which can have any number of bits between 0 and $2^{39} - 256$.

The output is a ciphertext *C* of the same length as *P*. The decryption process takes the inputs *C*, *K*, and *IV* and either outputs *P* or $\perp$, depending on whether there was tampering on the cloud platform or during message transmission.

```

pointcut methodCalls():                               2
    execution(* QueensBehaviour.*(..));                3
                                                        4
Object around(): methodCalls() {                      5
    ClassPool clPool=ClassPool.getDefault();          6
                                                        7
    byte[] initHash =                                  8
        (byte[])codeHash.get("QueensBehaviour");      9
    byte[] hash;                                       10
    try {                                              11
        CtClass cls =                                 12
            clPool.get("QueensBehaviour");             13
        byte[] bytecode = cls.toBytecode();           14
        MessageDigest messageDigest =                15
            MessageDigest.getInstance("SHA-256");      16
        messageDigest.update(bytecode);               17
        hash = messageDigest.digest();                18
        return proceed();                             19
    }                                                  20
    finally {                                          21
        if (java.util.Arrays.equals                  22
            (hash,initHash)) {                        23
            System.out.println("true_hash_value");    24
        }                                              25
        else {                                         26
            reportTamper();                            27
        }                                              28
    }                                                  29
}                                                       30

```

Listing 1 – AOP example using AspectJ.

7. Security analysis

In this section, we provide a security analysis of the proposed tamper-resistance model by discussing its resilience against various attacks listed in [Section 6.1.1](#).

7.1. Resilience against application code tampering

The proposed model provides protection against active data and code tampering attacks using software guards to check the dynamic integrity of program code. It is resistant against modification of agent code at load-time and hot-swapping of code during runtime. The guards detect tampering with agent code as soon as the code section that was tampered with is executed, which means the time of detection is dependent on the location (order) of the tamper in the program. In a more complex attack scenario, an adversary can tamper with the guard code protecting a specific code segment in addition to the code segment itself, in a way that will cause the application code tampering to go undetected. The resilience of the approach against kind of attack is described below.

7.2. Resilience against guard code tampering

For this attack, we consider an adversary who modifies any guard code in the agent to prevent the detection of tampering with the application code. If the adversary is able to successfully suppress the tamper detection and reporting mechanism, the agent will continue execution on the platform until program completion. Modification in the guard code during or before runtime will however cause an incorrect hash value for the corresponding guard to be integrated into the encryption key, which will result in a ciphertext that cannot be decrypted by the mobile platform. In order to get a faulty encryption key in case of any tampering with guard code, we need to ensure that the derivation process creates a secure (collision-resistant) key. Below we prove that the encryption key derived using the proposed tamper-resistance model is as secure as the output of the function used for obtaining guard hashes (i.e. SHA-256).

7.2.1. Proof of security of the derived encryption key

Let H be a collision-resistant hash function producing uniformly random output in the range $(0, 2^{256}-1)$, and T_1 and T_2 be two plaintexts. Given

$$H_1 = H(T_1) \quad (6)$$

$$H_2 = H(T_2) \quad (7)$$

$$H_3 = H_1 \oplus H_2 \quad (8)$$

We need to prove H_3 is uniformly random in the range $(0, 2^{256}-1)$ as well, where $\oplus$ is the bitwise XOR operator. Let $P_x(0)$ be the probability that the least significant bit of H_x is 0 and $P_x(1)$ be the probability that the same bit is 1. Following the uniform randomness property,

$$P_1(0) = 1/2, P_1(1) = 1/2 \quad (9)$$

$$P_2(0) = 1/2, P_2(1) = 1/2 \quad (10)$$

It follows from the properties of the XOR operator that:

$$P_3(0) = 1/2, P_3(1) = 1/2 \quad (11)$$

By applying the same arguments for the other bits as well, we prove that H_3 is uniformly random in the range $(0, 2^{256}-1)$. Therefore, the key generation algorithm produces output as secure as the SHA-256 secure hashing algorithm.

7.3. Resilience against communication tampering

The proposed approach uses an authenticated encryption scheme (GCM) for protecting the privacy and integrity of the result sent from an autonomous agent to the mobile platform. According to this scheme, an attacker would need to have knowledge of the secret shared key of the mobile platform and the autonomous agent in order to be able to impersonate the agent in a message sent to the mobile platform. Any message encrypted with a key other than the true shared key will not be decryptable by the mobile platform. Therefore, the proposed model provides protection against both man-in-the-middle attacks and any kind of tampering with the communication content between the mobile platform and the autonomous agents using authenticated encryption. In order to obtain the secret key of the mobile platform and the agent, an attacker can attempt to copy the agent, run it in its original form elsewhere, and upon discovering the key, encrypt a bogus result with the key to send back to the mobile platform. The proposed model provides defense against this type of attack as follows: In order to prevent replay attacks, where attackers can attempt to use execution flow of old agents, the integration of application sections (methods) into the hash calculation is randomized at each execution. In order to prevent dynamic code analysis attacks, i.e. to limit the exposure window of the agent, timeouts are set for the validity of the agent, after which the mobile platform terminates communication with the agent and spawns another agent for that module. Also, for an attacker to be able to send a bogus result to the mobile platform, the message has to be sent on the same bridge established upon the call, as the mobile platform will only accept messages on this bridge as valid. In order to strengthen the security of the result, an ensemble method offloading multiple copies of the agent can also be utilized as described below.

7.4. Sophisticated attacks tampering with the result

Let us assume the presence of an adversary in the agent's environment with exact knowledge of the operation principles of the agent, such that it can derive the secret key of the agent by actually running the agent code. Let us further assume that the adversary intercepts agent communication with the mobile platform and replaces the computation result with a bogus one encrypted with the correct secret key. Under such a sophisticated attack, the mobile platform will not be able to detect the presence of tampering with the decryption process.

To provide resilience against this type of attack, the proposed approach can be extended to offload multiple copies of the agent to the cloud instead of just one. The correctness of the computation result in that case will be judged based on a comparison of the results sent by the different agents. In the absence of collusion attacks, different results would signify tampering with at least one of the agents, in which case the results would not be integrated by the mobile platform. Actually, only two copies of an agent will be sufficient to judge correctness as shown below.

Proof that two agent copies are sufficient for judgement of correctness in the absence of collusion attacks:

There are four possible cases when running two copies A1 and A2 of the agent on two different cloud hosts:

1. The result from A1 is correct and that from A2 is faulty.
2. The result from A2 is correct and that from A1 is faulty.
3. The results from A1 and A2 are faulty and different.
4. The results from both A1 and A2 are correct.

The other possible case, where both results are faulty and are the same as each other is discarded here, as that most likely requires collusion of the attackers on the two platforms. In the first three cases the results will be rejected, as they will be different from each other. The only case when the results can be integrated with assurance of correctness will be the last one. Therefore, two copies of the agent will be sufficient to rule a result as untrusted. Note that this approach is complementary to the main approach rather than competing, as the main approach enables early detection of tamper in the case of no tampering with guard code and ensures that the results are coming from the correct platform (authenticity) and running of multiple copies does not have these effects by itself.

In addition to the tampering attacks against the application and guard code, attackers resident on the cloud host to which the agent migrates or the cloud host itself may try to tamper with the agent code. The agent code will be secure initially, as the agent originates from the mobile platform. Additionally, the agent will have to execute the correct application code and guard code in order to generate the correct encryption key for the result. In order to ensure that the agent code has not been modified in transit to the cloud, the tamper resistance process starts with an integrity check of the whole agent upon arrival at the cloud host. Therefore, dynamic code analysis by malicious parties on the cloud host will be needed to modify the agent's behavior to send a bogus response to the mobile platform, and the defense approaches described in Sections 7.3 and 7.4 can be utilized to mitigate such attacks.

8. Tamper resistance overhead

We also performed experiments to evaluate the performance overhead of the proposed tamper-resistance mechanism for the makespan of the two applications introduced above.

8.1. Experiments with face recognition

The experiments with the face recognition application were performed for the case of a picture database of 32 subjects

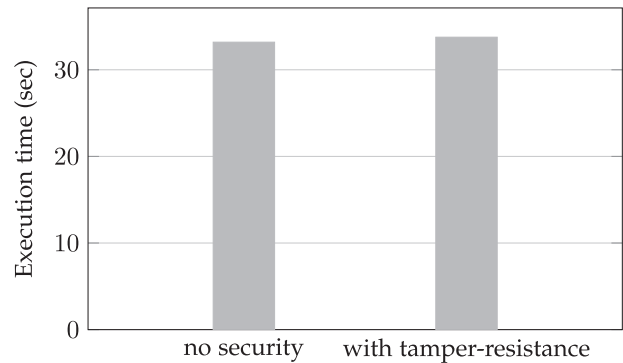


Fig. 7 – Comparison of execution times for the face recognition application with vs. without tamper-resistance mechanism.

to compare against. An Android emulator was used as the mobile platform, and a medium VMI in the west1-a region of EC2 was used to host the mobile agents. 62 integrity checkpoints were inserted into the agent code. Each experiment was repeated 75 times and we report the average makespans. Fig. 7 shows the comparison of the average makespan of the face recognition application with code offloading in the case of no tamper-resistance mechanism present and the average makespan of the same application with the proposed tamper-resistance mechanism integrated.

As seen in the figure, the difference between the average makespans for the two cases is always less than 600 ms, which is only about 1.8% of the total execution time. The average makespan for on-device execution for the same application is around 255 sec, therefore AAMCC with tamper resistance still performs much better than monolithic execution on a mobile device, and the tamper resistance overhead is negligible. The size of the agent code in the tamper-resistant case is 5 KB more than that in the case with no protection mechanism, which causes an additional delay of only 0.001 seconds on the Wi-Fi network.

8.2. Experiments with NQueens puzzle

The experiments with the NQueens application were performed using a Motorola Atrix 4G device to host the mobile application, whereas a medium VMI in the west1-a region of EC2 was used to host the cloud platform. 2 integrity checkpoints were inserted into the agent code. Fig. 8 shows the comparison of the average makespan of the NQueens application with code offloading in the case of no tamper-resistance mechanism present and the average makespan of the same application with the proposed tamper-resistance mechanism integrated, for different number of queens.

As seen in the figure, the difference between the average makespans for the two cases is always less than 2.2 s, which is less than 3.6% of the total execution time for the case with 15 queens. The average makespan for on-device execution for the same case is 872 s, therefore the AAMCC with tamper detection still performs much better than execution on the mobile device, and the tamper resistance overhead is negligible once again. The size of the agent code in the tamper-resistant

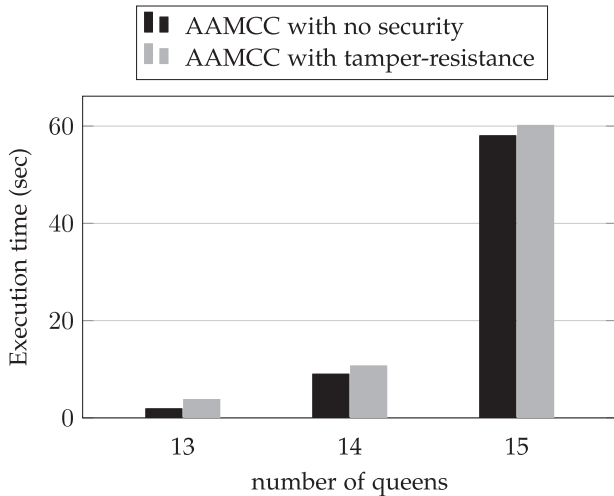


Fig. 8 – Comparison of execution times for the NQueens solver application with vs. without tamper-resistance mechanism.

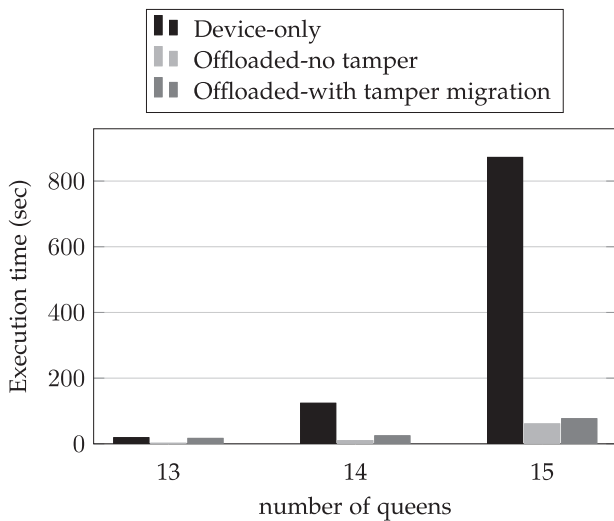


Fig. 9 – Comparison of execution times for the NQueens application for (a) device-only execution, (b) offloaded execution with no tampering, and (c) offloaded execution with tampering on first VMI.

case is 3 KB more than that in the case with no protection mechanism, causing an additional delay of only 0.0006 s on the Wi-Fi network.

We also performed experiments where the first VMI to which the NQueens application module was offloaded had a faulty class loader, i.e. the agent code was tampered with at the first hop it was offloaded to. Fig. 9 shows the comparison of the execution times for (a) device-only execution, (b) offloaded execution with no tampering, and (c) offloaded execution where execution on the first VMI the agent runs on involves tampering, but the next VMI it migrates to after detection of tampering (at the first integrity checkpoint) does not. The execution time for the offloading case is measured as the

time between the start of the application and the time when a correct result is received from the agent.

Fig. 9 shows that execution with tampering (involving migration of the agent) involves a fixed extra cost of about 15 seconds over the no-tamper case, due to the migration of the agent, for all numbers of queens. However, even with tampering involved, AAMCC with tamper resistance still achieves significantly higher performance than device-only execution.

9. Related work

Significant research efforts have been put into computation offloading since the early 1990s. Improvements in virtualization technology, availability of high network bandwidths and cloud computing infrastructures increased the feasibility of real-time computation offloading from mobile to cloud platforms in the past decade.

Early work in dynamic mobile-cloud computing models includes CloneCloud (Chun et al., 2011) and MAUI (Cuervo et al., 2010), both of which partition applications using a framework that combines static program analysis with dynamic program profiling, and optimizes execution time and energy consumption on the mobile device using an optimization solver. The disadvantage of these approaches is that they require a copy of the whole application code/virtual machine at the remote execution site, which is both a quite strict requirement for public cloud machines and makes the application code vulnerable to analysis by malicious parties on the same platform. Yang et al. (2012) propose a partitioning and execution framework specifically for mobile data stream applications. Huang et al. (2012) propose a low-complexity offloading algorithm to minimize energy consumption on a mobile device, however they do not provide details of the system architecture. Lin et al. (2014) present a model for energy-aware task scheduling on mobile devices, where tasks are assigned to cores on the device or a cloud resource based on their precedence requirements. Their approach gives precedence to minimizing energy consumption rather than total execution time. The ThinkAir (Kosta et al., 2012) framework provides better scalability and parallelism features than its predecessors, however it still requires the existence of the complete application code on the cloud server, exhibiting the disadvantages of CloneCloud and MAUI. Furthermore, none of these approaches integrates a mechanism for protection against tampering.

The two main approaches to provide security in MCC are (a) ensuring the security of the cloud platform on which the mobile code will execute for all users of that platform and (b) ensuring the security of the mobile code and data sent to the cloud platform for execution, without relying on the trustworthiness of the cloud platform.

One approach taken by previous research (Brenner et al., 2011; Sander and Tschudin, 1998) in mobile code security and secret program execution is homomorphic encryption, which operates with encrypted functions obtained by homomorphic transformations. However this approach was only shown to be usable for polynomial and rational functions, which prevents it from providing a general solution to the problem of secret execution in the cloud. Another popular approach for

preventing malicious analysis of mobile code is obfuscation. A code obfuscator is a tool which repeatedly applies semantics-preserving code transformations to a program (Heffner and Collberg, 2004). The obfuscator tries to make a program as incomprehensible as possible in order to protect it from being reverse-engineered or to protect a secret stored in the program from being discovered. While obfuscation makes a program more difficult to analyze, it comes at a cost: The obfuscated programs usually have greater complexity than their original, optimized counterparts, which causes decreased performance.

Zhang et al. (2009) propose an authentication and secure communication framework for elastic applications, which consist of weblets (program partitions) running on mobile device and cloud nodes concurrently. Their proposed method leverages elasticity managers on the cloud and the mobile device to establish a shared secret between weblets, to provide authentication between the weblets. They also propose approaches to authorize weblets running on the cloud to access sensitive data such as those provided by other web services. While this is one of the few proposals considering secure mobile-cloud computing, the authors do not mention any implementation details or performance results.

Zhou and Huang (2012) propose a framework that offloads computationally-intensive encryption and decryption operations on a mobile device to the cloud by extending the ciphertext policy attribute-based encryption scheme, so that no information about the data contents and security keys is revealed. The main problem with this scheme is that the ciphertext grows linearly with an increasing number of attributes, which hurts real-time performance. Xiao and Gong (2010) propose a collaboration model between mobile and cloud platforms to dynamically generate credentials to protect the mobile user from different types of attacks. The proposed scheme assumes that the cloud is a fully trusted component in the implementation of the proposed security solution. This assumption is too strong especially for public clouds. Chow et al. (2010) propose a policy-based cloud authentication platform for mobile users, which is based on the application of hash functions on data by the mobile platform with a client-generated key and the transfer of the generated information to a data collector in the cloud. The main shortcoming of this approach is that the mobile client has to apply a hash function on frequently generated data to achieve privacy, resulting in high performance overhead.

Most of the security solutions for mobile-cloud computing listed above focus on the confidentiality of mobile data offloaded to the cloud. Frameworks for protecting the runtime integrity of code offloaded to the cloud are still at a mostly immature state. Although not tailored for MCC, one state-of-the-art approach for protecting the integrity of computation is the *software guards* algorithm proposed by Chang and Atallah (2002). This algorithm is based on the partitioning of program code into regions, where each region is either user code, a checker guard that checks the integrity of a code region/other guard code or a responder guard that replaces a tampered code region with the original code for that region, i.e. guards are responsible for protecting each other in addition to the application code. The level of protection provided by this approach depends on the number of guards integrated into the

program, where a greater number of guards covering a larger part of the code base will provide better protection against attacks. The two main issues with increasing the resilience of a software program against tampering attacks with this approach are that integration of an increased number of guards into the program code results in increased code size and the integration of guard code into the program could result in significantly lower performance, especially if the guard code dominates the program, making it unfit for real-time MCC.

There is also an extensive literature on the protection of mobile agents from malicious modification on untrusted hosts. Roth (2002) provides a list of approaches that enable detection of tampering with the integrity of agent data in multi-hop agent paths. Although the approaches can be extended for detecting tampering with agent code as well, they do not allow for early detection, i.e. tampering only becomes evident when the agent returns to its original host, limiting their use in case of real-time MCC. Roth also mentions possible use of trusted processors (Trusted Platform Module) to protect agent integrity, however the assumption of the existence of such architectures on the cloud platforms used is too limiting for wide applicability of the model. Vigna et al. (2002) propose an intrusion detection system for agents, based on the inspection of audit trails formed during the execution of each mobile agent. Their approach requires an agent server architecture specifically providing the audit functionality and mining the data gathered from the audit trails in real-time to detect malicious modification may not provide accurate results especially under resource-constrained environments. Karnik and Tripathi (2001) describe a security mechanism for the Ajanta mobile agent system, allowing for detection of data integrity violations by the agent's owner. Their approach is targeted for agent data protection and detection takes place upon return to the original host server, ruling out an early recovery possibility needed in real-time MCC. Jochen describes (Jochen et al., 2003) a tamper detection marking mechanism to integrate units similar to watermarks into agent code to detect modifications, which requires analysis of the code by the originating host.

Various solutions provide software-based attestation using trusted hardware such as Trusted Platform Module (TPM). A TPM is a hardware component that can facilitate various security operations, such as authentication, hardware-based encryption, digital signing, secure key storage, and attestation of installed software (Parno et al., 2010). TrustVisor is a specialized hypervisor that provides secure and isolated execution of security-sensitive operations of an application using a trusted computing base (TCB) and can also perform remote attestation of the isolated execution (McCune et al., 2010). Existence of trusted hardware components would be a strong assumption for MCC. A software-based attestation solution that relies on timing measurements for checksum calculation has been proposed where the detection occurs because of the tamper's negative impact on the measurement (Kovah et al., 2011).

A lot of recent work has focused on attestation for Internet of Things (IoT) devices. General hardware solutions like TPM are not applicable for the IoT devices because of limited hardware. SEDA is a remote attestation solution focusing on verifying the integrity of the large network of embedded devices as a whole (Asokan et al., 2015). It works by distributing the attes-

tation workflow across the entire network by propagating and aggregating the information along a hierarchical tree. However, it requires strong trust guarantees for all devices and only provides protection against remote software-based attacks. SMART is a hardware-based mechanism to establish a dynamic root of trust for attestation of code in embedded devices using a read-only non-volatile memory to store the secret key material that cannot be changed after deployment (Eldefrawy et al., 2012). Further advancements have been made to derive the minimal features required to support attestation for embedded devices (Francillon et al., 2014). However, strong assumptions such as the uninterrupted execution of the solution and the minimalistic approach may not be feasible for regular systems because the hardware changes may not be directly applicable. C-FLAT is another approach for remote attestation of embedded devices that monitors the execution control-flow path of the software by leveraging the hardware trusted zones to detect any deviation (Abera et al., 2016). It instruments the control-flow instructions and computes the cumulative hash of the executed blocks to verify the validity of the execution. It is limited to detection of control-flow deviations and may not be able to detect run-time data-based attacks.

While most of the solutions for embedded devices do not consider physical attacks on devices, DARPA is a solution that addresses the detection of physical attacks (Ibrahim et al., 2016). It uses a heartbeat-based protocol requiring each device to periodically broadcast heartbeat messages which are used during attestation of a network of devices. It relies on the assumption that a compromised device goes offline for a considerable time in which it does not send the heartbeat message which can be used for detection. This can result in frequent false positives because of failures in network, devices and their software.

10. Conclusion

In this paper we proposed a mobile-cloud computation model based on autonomous agent-based application modules, which only requires the presence of isolated execution containers for the offloaded agents in the cloud, as opposed to other approaches requiring replication of full application code. We presented the design of a prototype of the proposed model with Android devices and cloud hosts on Amazon EC2 and showed through experiments that autonomous agent-based computation offloading is an effective tool for MCC, providing application execution times significantly lower than on-device execution. We discussed major security issues with offloading code and data to a public cloud and proposed a tamper-resistance model for MCC based on self-protecting agents, using various integrity checkpoints of code introspection to detect and report tamper. The proposed model mitigates the issues of end-to-end security, cloud multitenancy vulnerabilities and tampering attacks in MCC by making offloaded code self-aware both with the use of self-integrity and self-performance checkpoints. The integration of these checkpoints is performed in a transparent manner from the software development perspective, through utilization of aspect-oriented programming. The employment of mobile agents in

the model enables automated migration of tasks between clouds, decreasing the burden on the mobile platform. The agents are autonomous in the sense that they monitor their own integrity and performance, according to which they make migration decisions without the need for management by their originating mobile platform.

We also discussed details of a state-based programming model for MCC, where the agent can continue execution from the last integrity-preserving checkpoint after migration to a different platform, instead of starting execution from the beginning. The proposed tamper-resistance model was shown to incur very low runtime overhead and successfully detect load-time and runtime code tampering attacks. The mobile-cloud computation model presented in this paper is promising to meet high-performance computing and integrity assurance needs in real-time mobile-cloud computing.

REFERENCES

- Abera T, Asokan N, Davi L, Ekberg JE, Nyman T, Paverd A, Sadeghi AR, Tsudik G. C-flat: control-flow attestation for embedded systems software. *Proceedings of the ACM conference on computer and communications security (CCS)*; 2016. p. 743–54.
- Ali M, Khan SU, Vasilakos AV. Security in cloud computing. *Inf Sci* 2015;305(C):357–83. Jun.
- Angin P, Bhargava B. An agent-based optimization framework for mobile-cloud computing. *J Wirel Mob Netw Ubiquitous Comput Dependable Appl* 2013;4(2):1–17.
- Angin P, Bhargava B, Jin Z. A self-cloning agents based model for high-performance mobile-cloud computing. *Proceedings of the IEEE international conference on cloud computing (CLOUD'15)*; 2015. p. 301–8.
- Asokan N, Brasser FF, Ibrahim A, Sadeghi AR, Schunter M, Tsudik G, Wachsmann C. SEDa: scalable embedded device attestation. *Proceedings of the ACM conference on computer and communications security (CCS)*; 2015. p. 964–75.
- AspectJ, <http://eclipse.org/aspectj/>. [Accessed: 5 January 2018].
- Bate I, Bernat G, Murphy G, Puschner P. Low-level analysis of a portable java byte code WCET analysis framework. *Proceedings of the seventh IEEE international conference on real-time computing systems and applications*; 2000. p. 39–46.
- Bellare M, Namprempre C. Authenticated encryption: relations among notions and analysis of the generic composition paradigm. *J Cryptol* 2008;21(4):469–91.
- Brenner M, Wiebelitz J, von Voigt G, Smith M. Secret program execution in the cloud applying homomorphic encryption. *Proceedings of the fifth IEEE international conference on digital ecosystems and technologies (DEST'11)*; 2011. p. 114–19.
- Chang H, Atallah MJ. Protecting software code by guards. *Security and privacy in digital rights management*. Springer; 2002. p. 160–75.
- Chess D, Harrison C, Kershenbaum A. Mobile agents: are they a good idea?. In: Vitek J, Tschudin C, editors. *Mobile object systems towards the programmable internet*. Springer, 1997.
- Chou TS. Security threats on cloud computing vulnerabilities. *Int J Comput Sci Inf Technol* 2013;5(3):79–88.
- Chow R, Jakobsson M, Masuoka R, Molina J, Niu Y, Shi E, Song Z. Authentication in the clouds: a framework and its application to mobile users. *Proceedings of the ACM workshop on cloud computing security*; 2010. p. 1–6.
- Chun B, Ihm S, Maniatis P, Naik M, Patti A. Clonecloud: elastic execution between mobile device and cloud. *Proceedings of*

- the sixth ACM european conference on computer systems (Eurosys'11); 2011. p. 301–14.
- Collberg C, Nagra J. *Surreptitious software: obfuscation, watermarking, and tamperproofing for software protection*. Addison-Wesley Professional; 2009.
- Cuervo E, Balasubramanian A, Cho D, Wolman A, Saroiu S, Chandra R, Bahl P. Maui: making smartphones last longer with code offload. *Proceedings of the eighth ACM international conference on mobile systems, applications, and services (mobisys'10)*; 2010. p. 49–62.
- Darnok, K. (2018). <http://darnok.org/programming/face-recognition>. [Accessed January 07].
- Eldefrawy K, Tsudik G, Francillon A, Perito D. Smart: secure and minimal architecture for (establishing dynamic) root of trust.. *Proceedings of NDSS*, 2012.
- Feng Z, Bai B, Zhao B, Su J. Shrew attack in cloud data center networks. *Proceedings of seventh international conference on mobile ad-hoc and sensor networks, (MSN)*; 2011. p. 441–5.
- Francillon A, Nguyen Q, Rasmussen KB, Tsudik G. A minimalist approach to remote attestation.. *Proceedings of DATE*; 2014. p. 1–6.
- Grew, D. M., & Viega, J. (2004). The galois/counter mode of operation (GCM). NIST Special Publication 800-38DNIST. GSM Arena. Motorola Atrix 4G. (2018). https://www.gsmaarena.com/motorola_atrx_4g-3708.php. [Accessed January 05].
- Heffner K, Collberg C. *The obfuscation executive*. Information security. Springer; 2004. p. 428–40.
- Huang D, Wang P, Niyato D. A dynamic offloading algorithm for mobile computing. *IEEE Trans Wirel Commun* 2012;11(6):1991–5. June.
- Ibrahim A, Sadeghi AR, Tsudik G, Zeitouni S. DARPA: device attestation resilient to physical attacks.. *Proceedings of wiSEC. ACM*; 2016. p. 171–82.
- Amazon Web Services Inc., Amazon Web Services Inc. Amazon elastic compute cloud. <http://aws.amazon.com/ec2>. [Accessed: 7 January 2018].
- Jochen M, Marvel L, Pollock L. A framework for tamper detection marking of mobile applications. *Proceedings of the international symposium on software reliability engineering (ISSRE)*; 2003. p. 143–53.
- Karnik NM, Tripathi AR. Security in the Ajanta mobile agent system. *Softw Pract Exp* 2001;31:301–29.
- Katz J, Yung M. Unforgeable encryption and chosen ciphertext secure modes of operation. *Fast software encryption*. Springer; 2001. p. 284–99.
- Kosta S, Aucinas A, Hui P, Mortier R, Zhang X. Thinkair: dynamic resource allocation and parallel execution in the cloud for mobile code offloading. *Proceedings of the thirty first IEEE international conference on computer communications (INFOCOM'12)*; 2012. p. 945–53.
- Kovah X, Kallenberg C, Weathers C, Herzog A, Albin M, Butterworth J. New results for timing-based attestation.. *Proceedings of the IEEE symposium on security and privacy (s&p)*; 2011. p. 239–53.
- Telecom Italia Lab, Java agent development framework. <http://jade.tilab.com/>. [Accessed: 7 January 2018].
- Lin X, Wang Y, Xie Q, Pedram M. Energy and performance-aware task scheduling in a mobile cloud computing environment. *Proceedings of the seventh IEEE international conference on cloud computing (CLOUD'14)*. IEEE; 2014. p. 192–9.
- McCune JM, Li Y, Qu N, Zhou Z, Datta A, Gligor VD, Perrig A. Trustvisor: efficient TCB reduction and attestation.. *Proceedings of the IEEE symposium on security and privacy (S&P)*; 2010. p. 143–58.
- Trusted Platform Module, Trusted Platform Module. http://www.trustedcomputinggroup.org/resources/trusted_platform_module_tpm_summary. [Accessed: 5 January 2018].
- Pahlsson, N. (e). Aspect-oriented programming. 2002 <http://oberon2005.oberoncore.ru/paper/np2002.pdf>. [Accessed: 5 January 2018].
- Parno B, McCune JM, Perrig A. Bootstrapping trust in commodity computers.. *Proceedings of the IEEE symposium on security and privacy (s&p)*; 2010. p. 414–29.
- Ristenpart T, Tromer E, Shacham H, Savage S. Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds. *Proceedings of the sixteenth ACM conference on computer and communications security*; 2009. p. 199–212.
- Roth V. Programming satan's agents. *Electron Notes Theor Comput Sci* 2002;63:124–39.
- Sander T, Tschudin CF. *Protecting mobile agents against malicious hosts*. Mobile agents and security. Springer; 1998. p. 44–60.
- Vigna, G., Cassell, B., & Fayram, D., An intrusion detection system for aglets. *Proceedings of the sixth International Conference on Mobile Agents (MA '02)*, 2002, 64–77.
- Xiao S, Gong W. Mobility can help: Protect user identity with dynamic credential. *Proceedings of the eleventh IEEE international conference on mobile data management (MDM'10)*; 2010. p. 378–80.
- Yang L, Cao J, Tang S, Li T, Chan AT. A framework for partitioning and execution of data stream applications in mobile cloud computing. *Proceedings of the fifth IEEE international conference on cloud computing (CLOUD'12)*; 2012. p. 794–802.
- Zhang X, Schiffman J, Gibbs S, Kunjithapatham A, Jeong S. Securing elastic applications on mobile devices for cloud computing. *Proceedings of the ACM workshop on cloud computing security*; 2009. p. 127–34.
- Zhou Z, Huang D. Efficient and secure data storage operations for mobile cloud computing. *Proceedings of the eighth IEEE international conference on network and service management (CNSM'12)*; 2012. p. 37–45.

Pelin Angin is an assistant professor of Computer Engineering at Middle East Technical University, Turkey. She received her BS degree in Computer Engineering at Bilkent University in 2007 and her PhD degree in Computer Science at Purdue University in 2013. Her research interests lie in the fields of mobile-cloud computing, cloud computing security, distributed systems and data mining.

Bharat Bhargava is a professor of the Department of Computer Science with a courtesy appointment in the School of Electrical and Computer Engineering at Purdue University. His research focuses on security and privacy issues in distributed systems. He is a Fellow of the Institute of Electrical and Electronics Engineers and of the Institute of Electronics and Telecommunication Engineers. In 1999, he received the IEEE Technical Achievement Award for his decade long contributions to foundations of adaptability in communication and distributed systems. He serves on seven editorial boards of international journals. He also serves the IEEE Computer Society on Technical Achievement Award and Fellow committees.

Rohit Ranchal is a researcher and technical lead in the Watson Health business unit at IBM. Dr. Ranchal received his PhD from Purdue University in 2015 where he was affiliated with Computer Science and CERIAS. His research interests include cloud and service computing systems with a focus on resilient, secure architectures, and healthcare solutions.